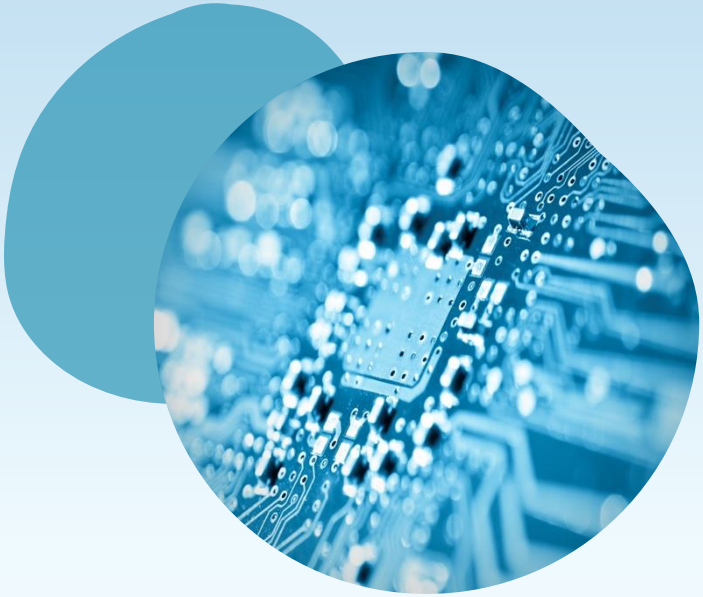


Embedded C

Embedded Software for the Internet Of Things

Kasim Sinan Yildirim

Software Independence

- Maximize Software **portability** and **reusability**
  - Attempt to write as much software with
    - Architecture Independence
    - Platform Independence
- Impossible to make everything independent
  - Firmware Layers still interact with hardware
  - Assembly is Architecture Dependent



ESP32

ESP32 Executable Program

Peripheral Module

GPIO HW



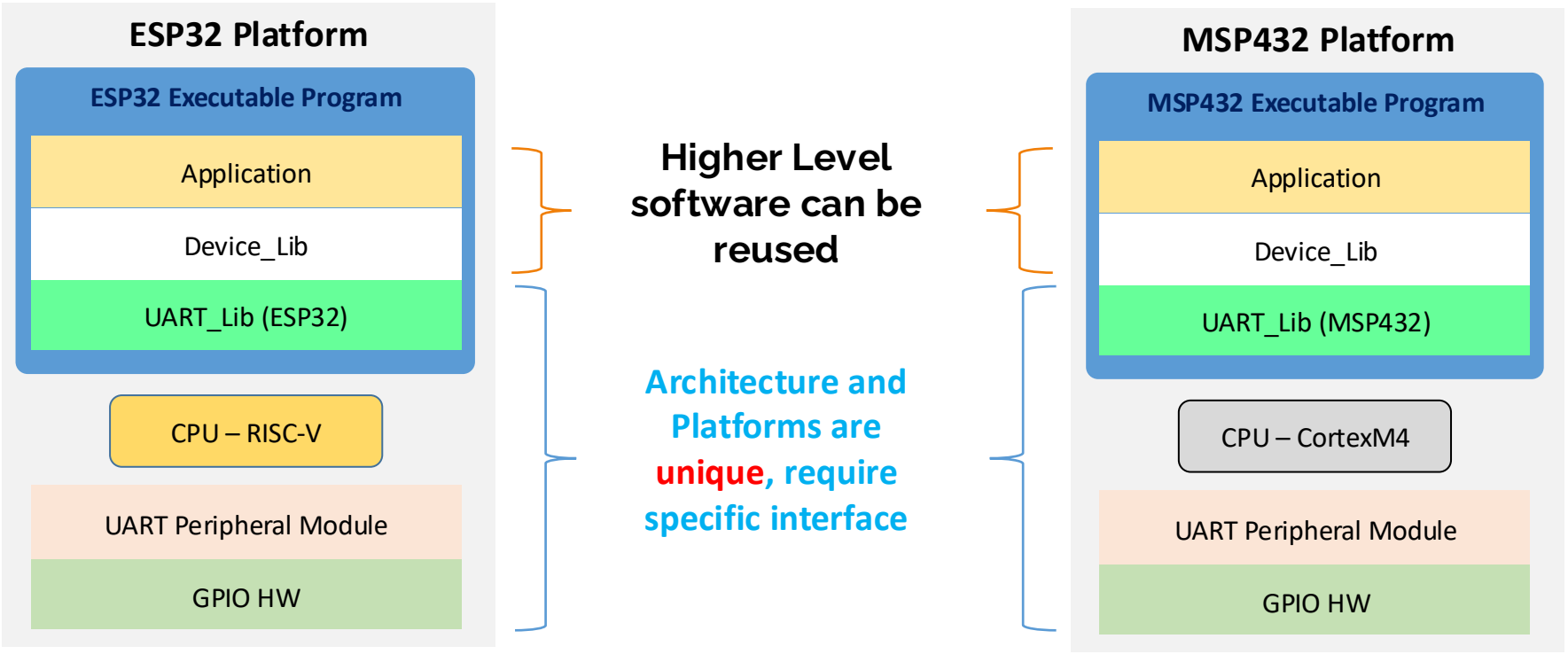
MSP432

MSP432 Executable Program

Peripheral Module

GPIO HW

Software Independence - Module Structure

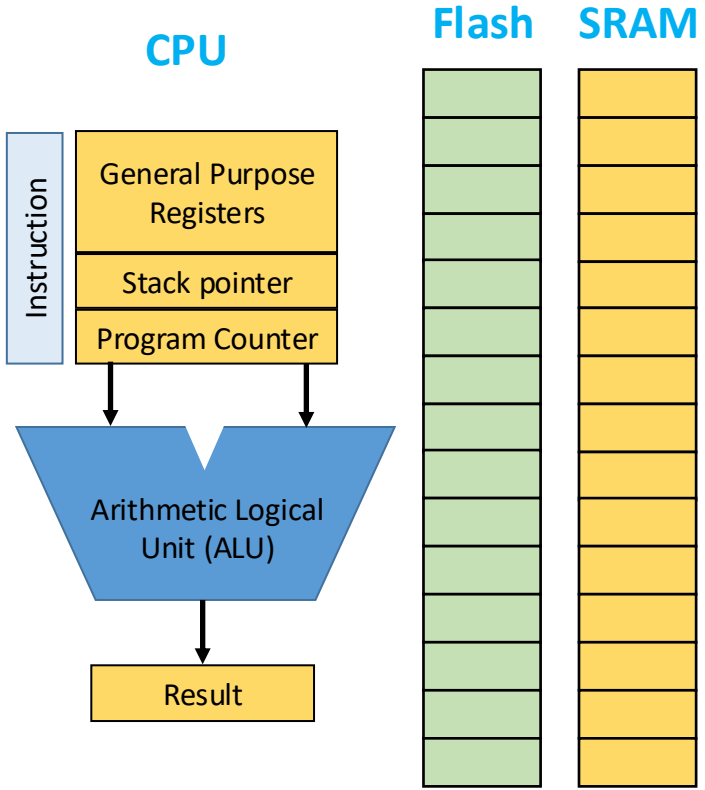


Architecture, ABI, and toolchain

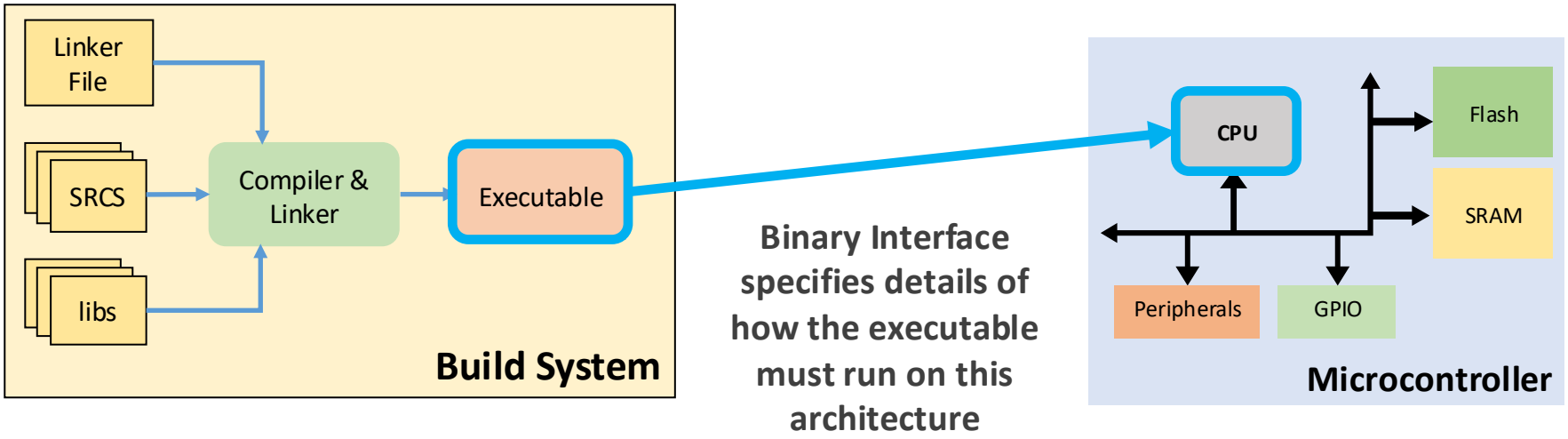
Layer	What it defines	Example dependency
C language	Syntax and abstract behavior	Data types and lifetime
Architecture	Instructions and address model	Arm Cortex-M versus RISC-V
ABI	Calling convention and data layout	Register arguments and structure alignment
Compiler	Extensions and optimization behavior	GNU attributes and pragmas
Linker script	Memory regions and section placement	Flash, SRAM, vectors, stack, and heap

Binary Interfaces

- **Application Binary Interface (ABI)**
  - Provides details on how a binary must be compiled and interfaced with platform components
    - Register Use / Word Size
    - Code/Data Storage Requirements
    - Addressing Modes
    - Calling Conventions
    - Helper Functions & Libraries

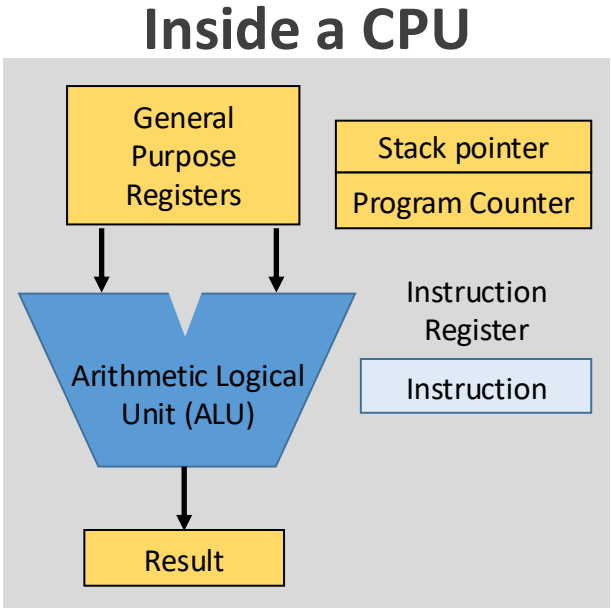


Binary Interfaces



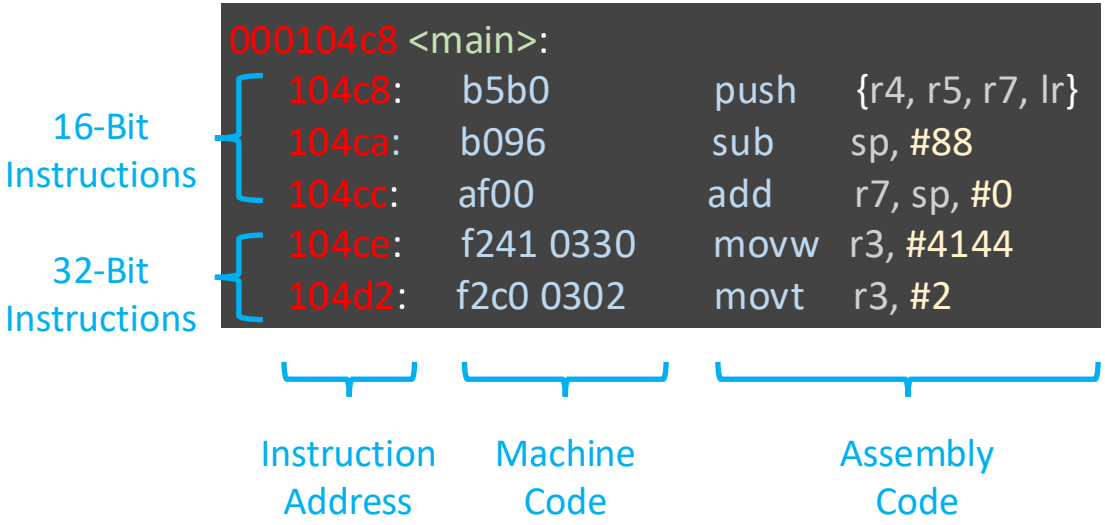
Architecture

- **Instruction** – Fundamental unit of work or operation
  - Arithmetic
  - Logical
  - Program Flow Control
  - Load/Store
- **Word** – fundamental operand size for each operation



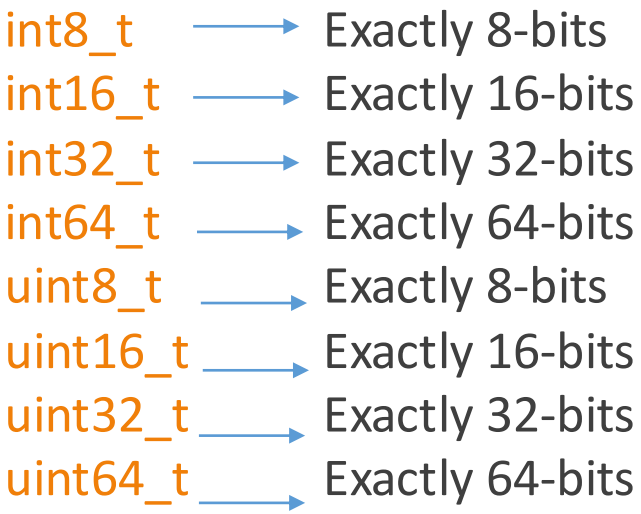
Instruction Sizes

- Instruction size can vary
  - ARM Instruction Set **ARMv6-M** -> 16-Bit and 32-Bit
  - Thumb-2 Instruction Set -> 16-Bit



Standard Integer Sizes

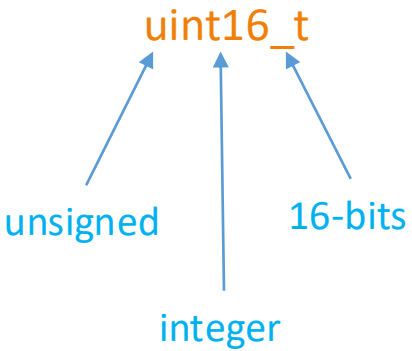
- Variable length Types can cause portability issues
- We want portable data types!
  - Explicitly defined types that specify storage and sign
  - Defined in the **stdint.h** header file

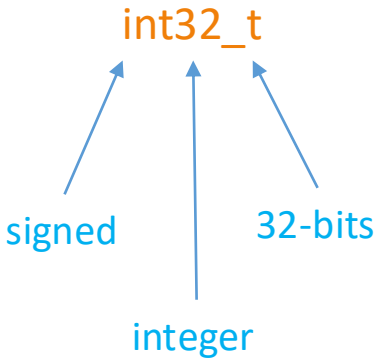


Standard Integer Sizes

- Explicitly defined types that specify exact size and sign
  - u = Unsigned
  - int = Integer type
  - _t = type

```
uint8_t   var1;
int32_t   var3 = -100;
```






```
#ifndef   __STDINT_H__
#define   __STDINT_H__

/* 8-bit signed/unsigned Integers */
typedef signed     char int8_t;
typedef unsigned char uint8_t;

/* 16-bit signed/unsigned Integers */
typedef signed     short int int16_t;
typedef unsigned short int uint16_t;

/* 32-bit signed/unsigned Integers */
typedef signed     long int int32_t;
typedef unsigned long int uint32_t;

/* 64-bit signed/unsigned Integers */
typedef signed     long long int int64_t;
typedef unsigned long long int uint64_t;

#endif /* __STDINT_H__ */
```

Typedef

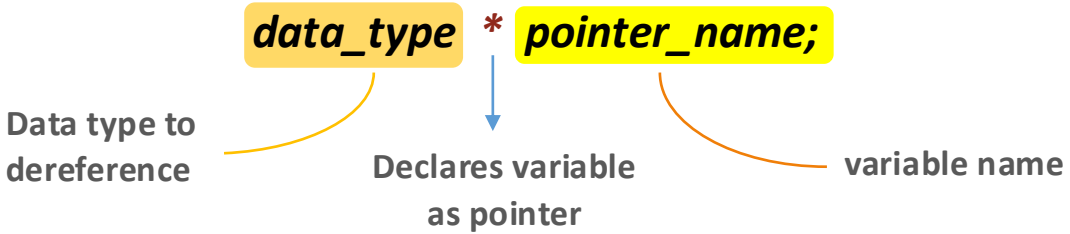
- **typedef** keyword allows programmer to create own types
  - like an alias
- Can apply to standard types or derived types

```
typedef enum Color {
    COLOR_BLUE = 0,
    COLOR_RED = 1,
    COLOR_GREEN = 2,
} Color_t;
```

```
typedef struct Data {
  int32_t  temperature;
  uint32_t date;
  uint32_t time;
} Data_t;
```

Pointer Types

- Pointer type denotes the data type that a pointer will **dereference**



Pointer Types

- Pointer type denotes the data type that a pointer will **dereference**



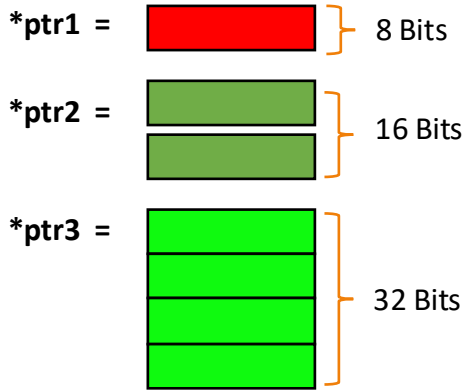
data_type * *pointer_name;*

Data type to dereference

Declares variable as pointer

variable name

```
uint8_t *ptr1 = (uint8_t *) 0x00;
uint16_t *ptr2 = (uint16_t *) 0x04;
uint32_t *ptr3 = (uint32_t *) 0x08;
```



Pointer Size

- All Pointers are the **same** length

sizeof(uint8_t*) = sizeof(int16_t*) = sizeof(uint32_t*) = sizeof(float*) = **32-Bits**!

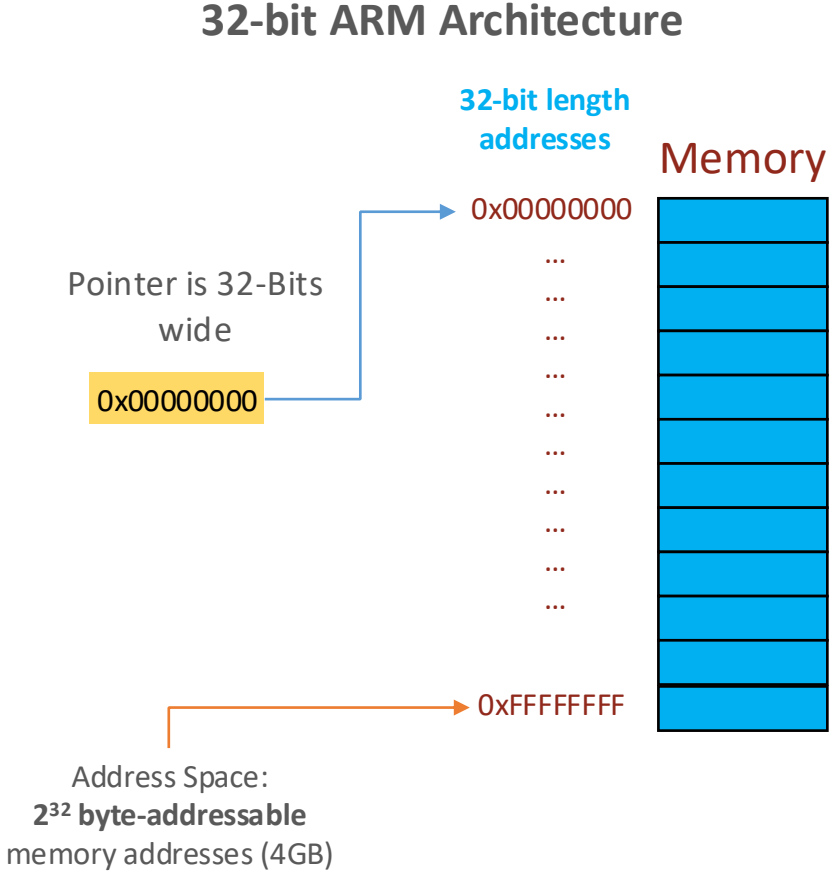
```
uint8_t  *ptr1    = (uint8_t *) 0x00;
uint16_t * ptr2   = (uint16_t *) 0x04;
uint32_t * ptr3   = (uint32_t *) 0x08;
float    * ptr4   = (float *) 0x0C;
```

sizeof(ptr1) = sizeof(ptr2) = sizeof(ptr3) = sizeof(ptr4) = 32-Bits!

sizeof(*ptr1)	≠	sizeof(*ptr2)	≠	sizeof(*ptr3)	≠	sizeof(*ptr4)
1 Byte		2 Byte		4 Byte		4 Byte

Pointer Size

- Pointers hold addresses to a location in memory
  - ARM -> 32-bit length
  - Address Space = $2^{\text{PointerLength}}$



Pointer Operators

- Dereference Operator = *
  - Accesses data at address
- Address-of Operator = &
  - Provides address of variable
- Integers are not addresses
  - Cast to explicit address for **Peripheral Memory**

```
uint32_t  var;
uint32_t * ptr = &var;
```

```
uint16_t * ptr = (uint16_t *) 0x480C0000;
```

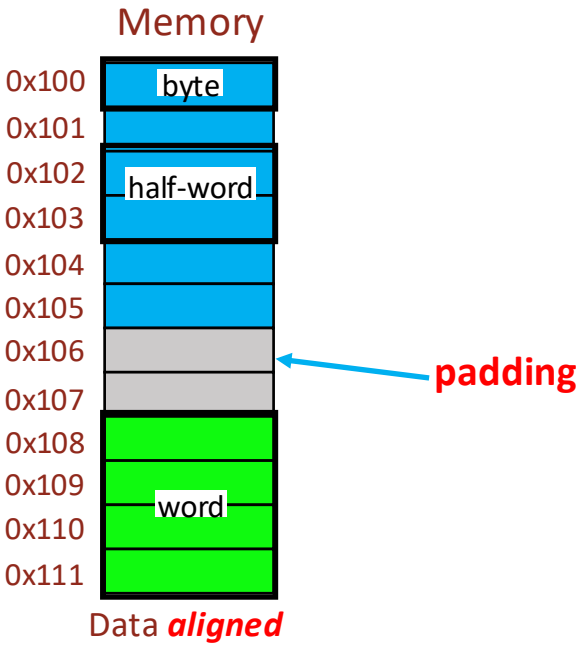
Casts Integer to address

Null Pointers

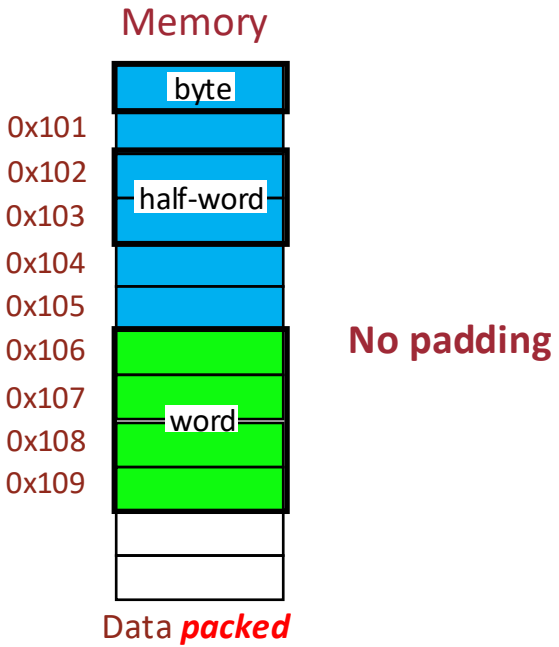
- At time of pointer declaration, you might not know the address
  - Using uninitialized pointer can potentially corrupt your memory
- Null Pointers point to nothing
  - Used to check for valid pointer
  - Dereferencing a NULL Pointer can cause an exception

```
#define NULL ((void*)0)
uint32_t * ptr = NULL;
...
if (ptr == NULL) {
    /* error! */
}
*ptr = 0xABCD1234;
```

Data Alignment

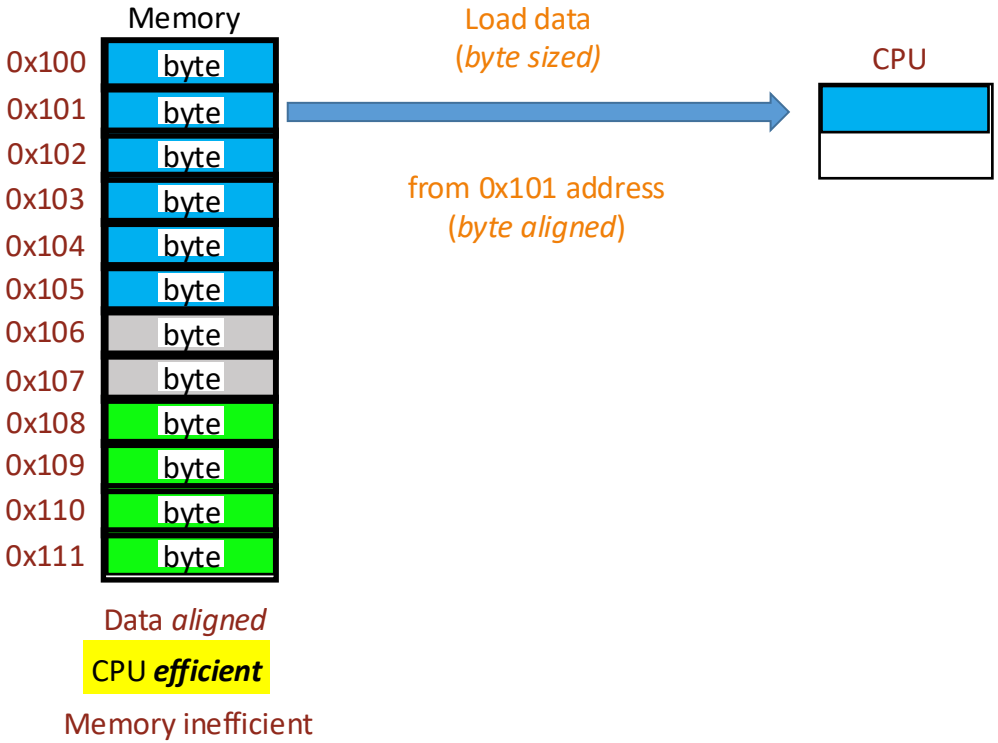


CPU **efficient**/
Memory inefficient



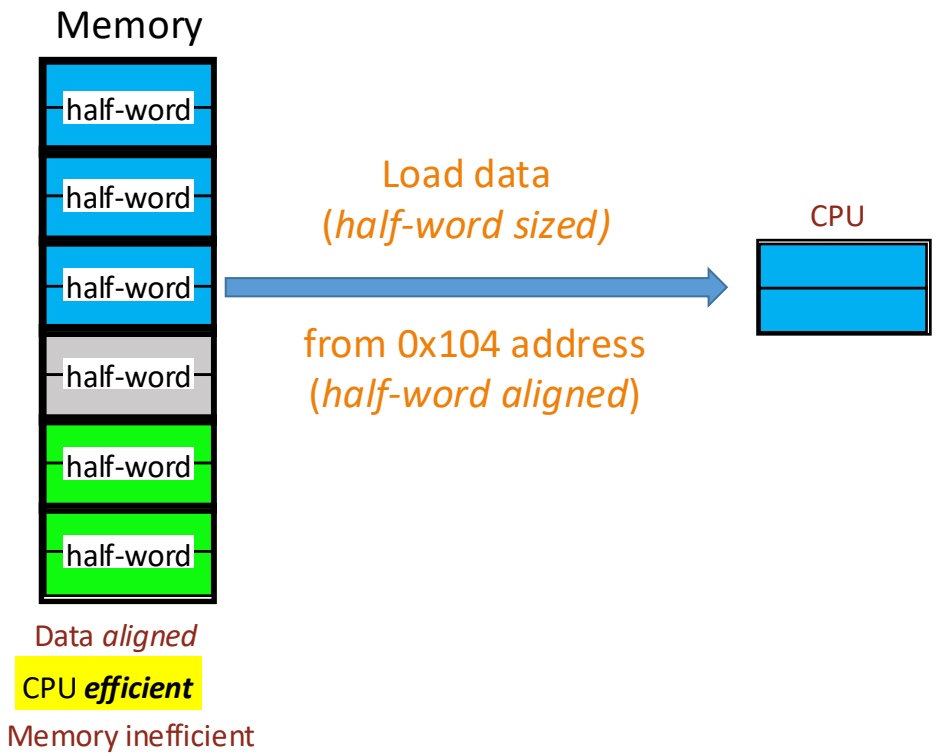
Memory **efficient**/
CPU inefficient

Load/Stores



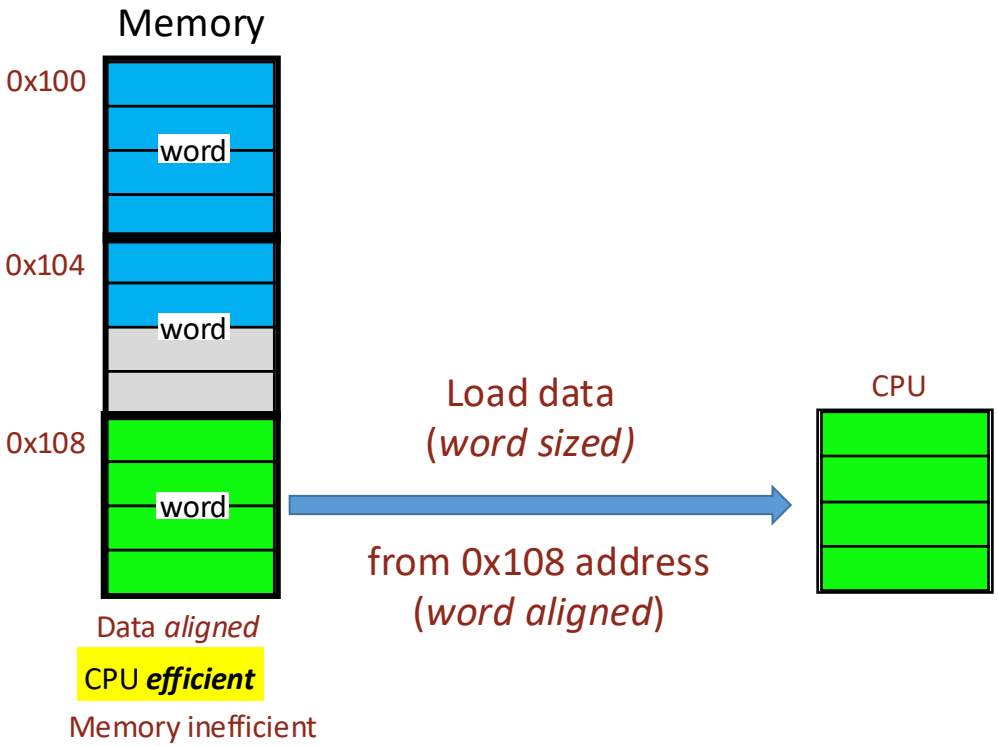
Load/store data occurs only at ***aligned addresses in memory***

Load/Stores



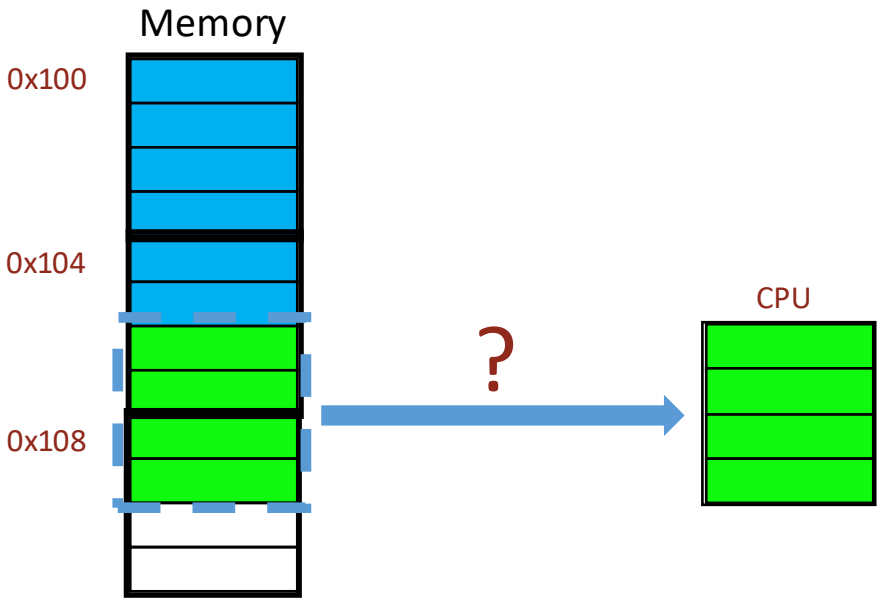
Load/store data occurs only at
aligned addresses in memory

Load/Stores



Load/store data occurs only at
aligned addresses in memory

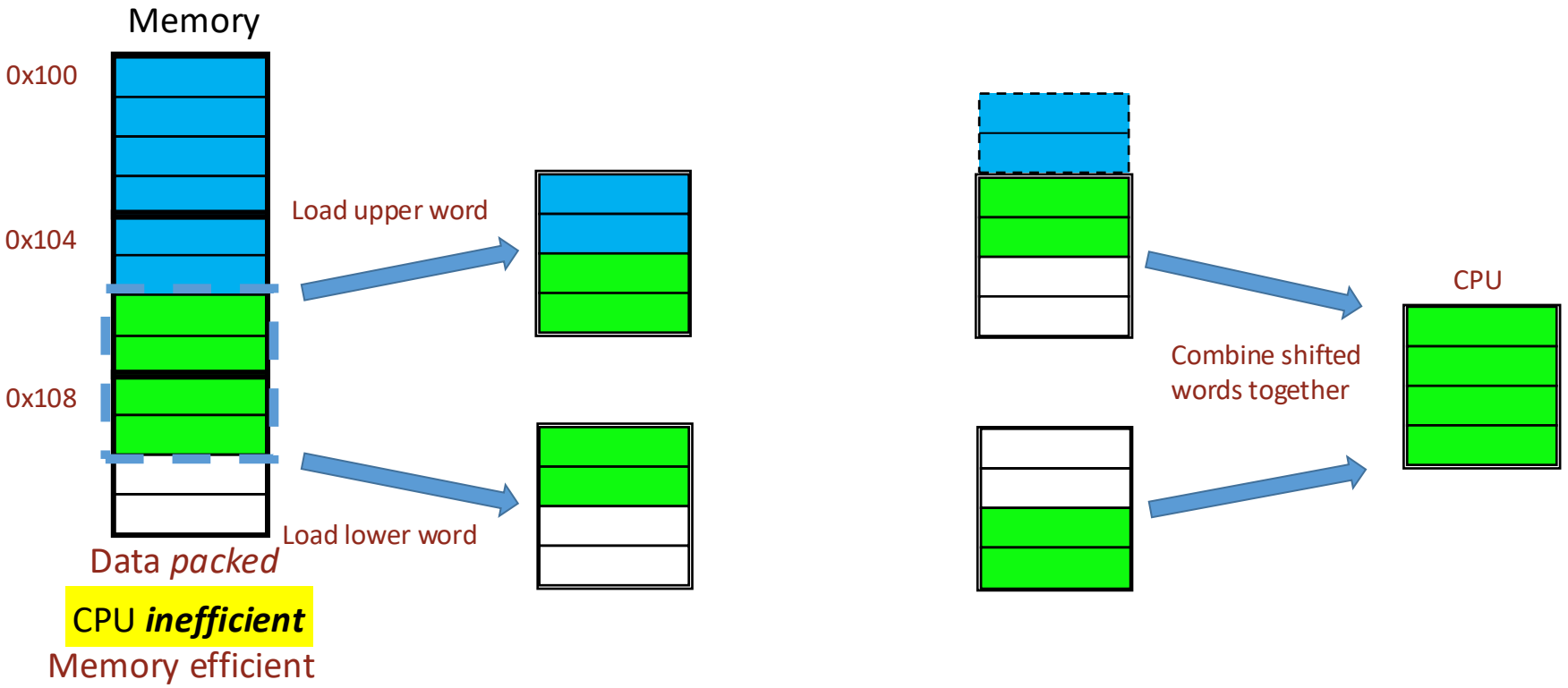
Unaligned Load/Store



Load/store data occurs only at
aligned addresses in memory

How to load **misaligned data**

Unaligned Load/Store

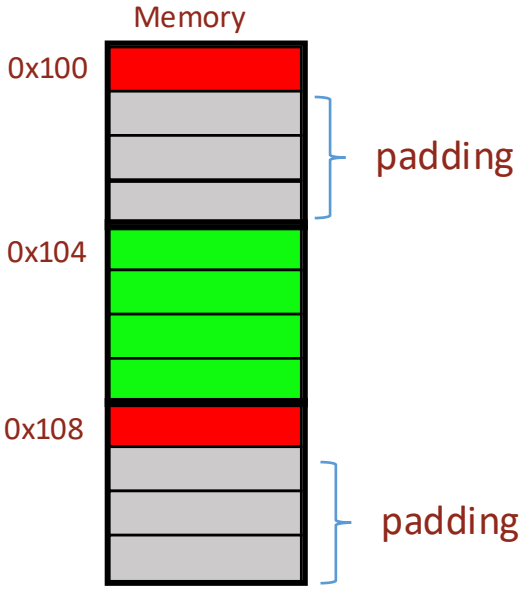


Struct in Memory

- Compiler attribute
  - **packed**

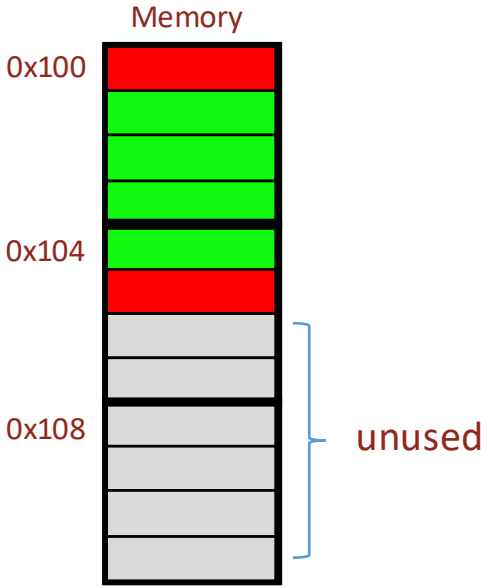
```
struct struct_name {
  int8_t var1;
  int32_t var2;
  int8_t var3;
};
```

```
struct struct_name {
  int8_t   var1;
  int32_t var2;
  int8_t   var3;
} __attribute__ ((packed));
```



Data *padded*

Size of struct_name is 12 bytes



Data *packed*

Size of struct_name is 6 bytes

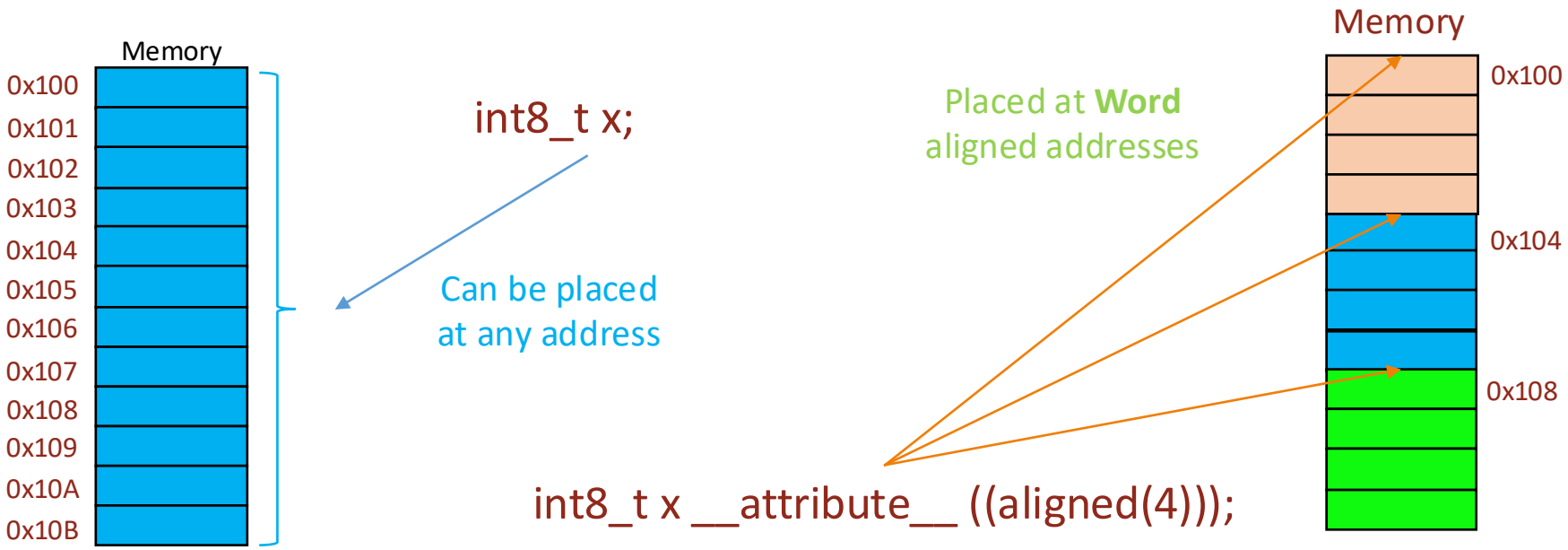
Compiler Attributes

- Attributes can give specific details on how to compile code for
  - Variables
  - Structures & Structure Members
  - Functions
- Attributes are **NOT** part of the C-standard
  - Not **Portable** Across Compilers

```
struct struct_name {
  int8_t   var1;
  int32_t var2;
  int8_t   var3;
} __attribute__ ((packed));
```

Alignment Attributes

- Alignment attributes specify memory alignment for data
  - Power of two: 2, 4, 8, 16



Alignment on a Structure

- Structures and structure members can be aligned

```
typedef struct  {
  int8_t   var1;
  int32_t var2;
  int8_t   var3;
} str1;
```



```
typedef struct {
  int8_t   var1 __attribute__ ((aligned(4)));
  int32_t  var2 __attribute__ ((aligned(4)));
  int8_t   var3 __attribute__ ((aligned(4)));
} str2;
```

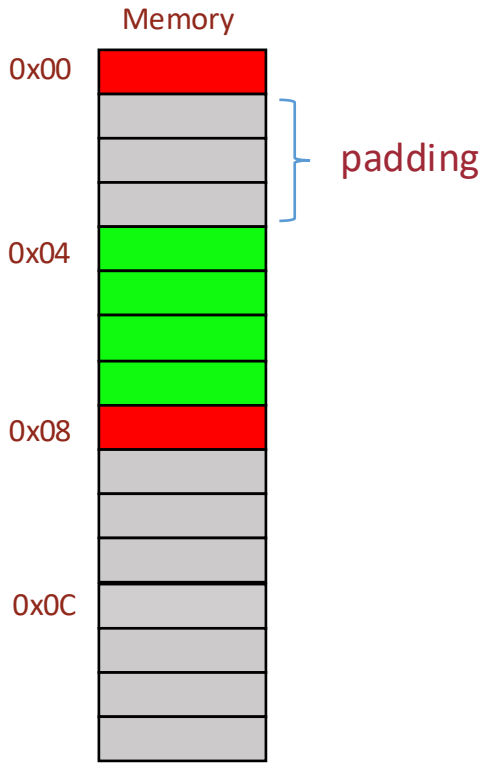
sizeof(str2) = **12 Bytes**

Alignment on a Structure

- Aligned structure members

```
typedef struct  {
  int8_t   var1;
  int32_t var2;
  int8_t   var3;
} str3 __attribute__ ((aligned));
```

sizeof(str3) = **9 Bytes**

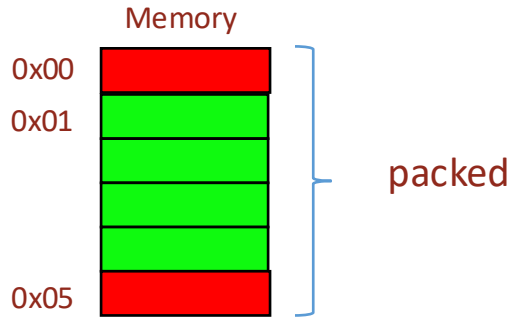


Alignment on a Structure

- Packed structure members

```
typedef struct  {
  int8_t   var1;
  int32_t var2;
  int8_t   var3;
} str3 __attribute__ ((packed));
```

sizeof(str3) = **6 Bytes**



Function Attributes

- Compiler attributes can apply to functions
  - Inline – Skips calling convention, **copies** function body into calling code
- Inline keyword is a c99 Feature
- always_inline is a **GCC attribute**
  - Not supported on other compilers

Good for **small** functions

Compiler will NOT ignore this

Compiler might ignore this

```
__attribute__ ((always_inline)) inline int32_t add( int32_t x, int32_t y )
{
    return (x + y);
}
```

Function Pragmas

- Pragmas provide **special instructions** to the compiler
  - Push/Pop – Add extra option for compilation
  - Optimize – Specify a certain level of optimization block of code

Only this code will have Zero Optimizations applied

```
#pragma GCC push
#pragma GCC optimize ("O0")
int32_t add( int32_t x, int32_t y )
{
    return (x + y);
}
#pragma GCC pop
```

Example: Check GNU C Support

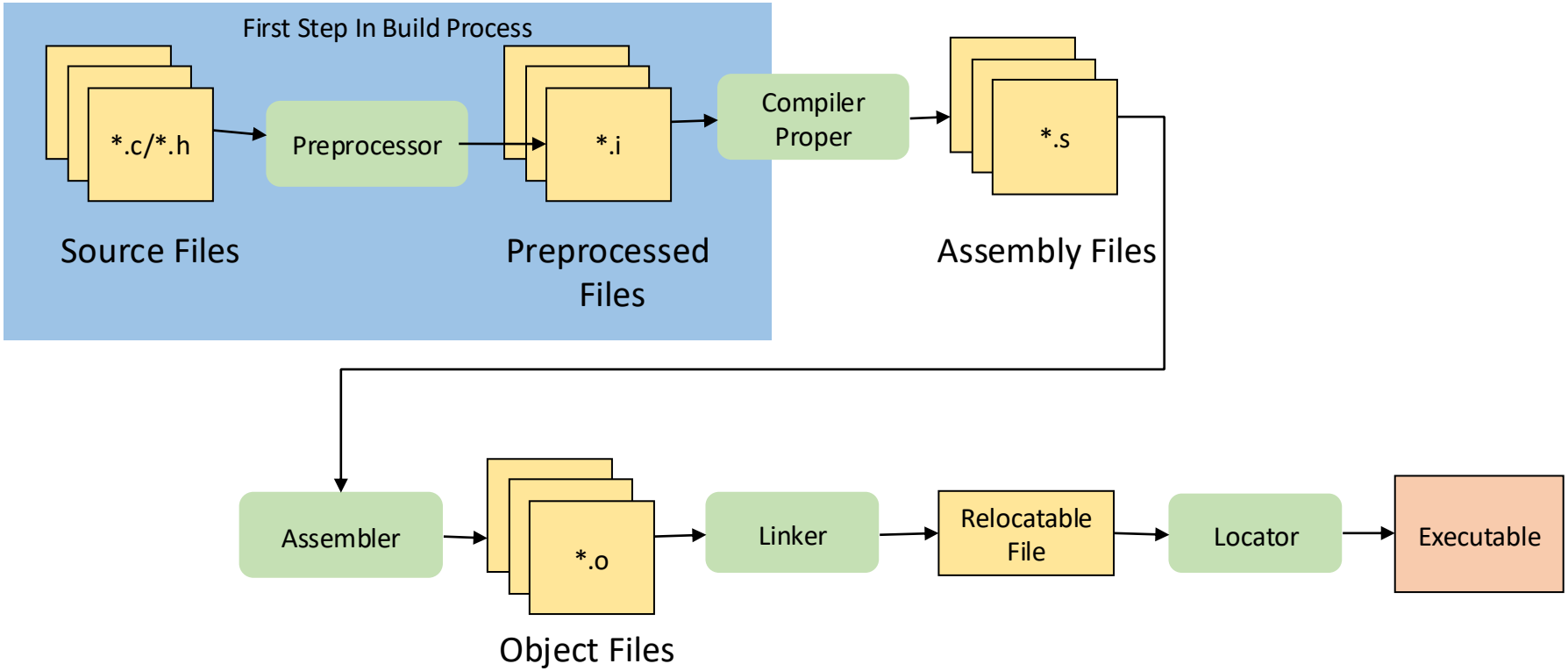
- Embedded teams can support multiple chipset platforms and multiple architectures
  - Different architecture may require different compiler
- E.g., __attribute__(x) is only a GCC compiler keyword
  - Throws errors for other compilers

```
#ifndef (__GNUC__)
#define __attribute__ (x)
#endif
```



Define nothing for Non-GNU C compilers

Preprocessor Directives



Preprocesor Directives

- Special keywords used by the preprocessor before compilation

- Directives start with ‘#’ sign
  - Important Directives
  - #define, #undef
  - #ifndef, #ifdef, #endif
  - #include
  - #warning, #error
  - #pragma

Preprocesor Output

- Stop after preprocessing
- Output the preprocessed file to a *.i extension

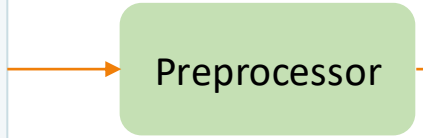
```
$ gcc -E -o main.i main.c
```

Macro Substitution

Original File

```
#define ZERO   0
#define LENGTH 10


int main(){
 char arr[LENGTH];
 memset(arr, ZERO, LENGTH);
    …
 return 0;
}
```



After Preprocessing

```
int main(){
 char arr[10];
 memset(arr, 0, 10);
    …
 return 0;
}
```

Macro Substitution

Original File

```
#define SQUARE(x) (x*x)

int main(){
 int y_sqrd;
 int y = 2;
 y_square = SQUARE(y);
 return 0;
}
```



After Preprocessing

```
int main(){
 int y_sqrd;
 int y = 2;
 y_square = (y*y);
    …
 return 0;
}
```

Macro Substitution

Original File

```
#define SQUARE(x) (x*x)

int main(){
 int y_sqrd;
 int y = 2;
 y_square = SQUARE(y++);
 return 0;
}
```



After Preprocessing

```
int main(){
 int y_sqrd;
 int y = 2;
 y_square = (y++*y++);
    …
 return 0;
}
```

Macro Substitution

After the line y_square executes:

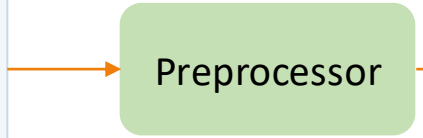
y = 4

y_square = 6

Original File

```
#define SQUARE(x) (x*x)

int main(){
 int y_sqrd;
 int y = 2;
 y_square = SQUARE(y++);
 return 0;
}
```



After Preprocessing

```
int main(){
 int y_sqrd;
 int y = 2;
 y_square = (y++*y++);
    …
 return 0;
}
```

Undefined Behavior!!!

Conditional Compilation (Compile Time Switch)

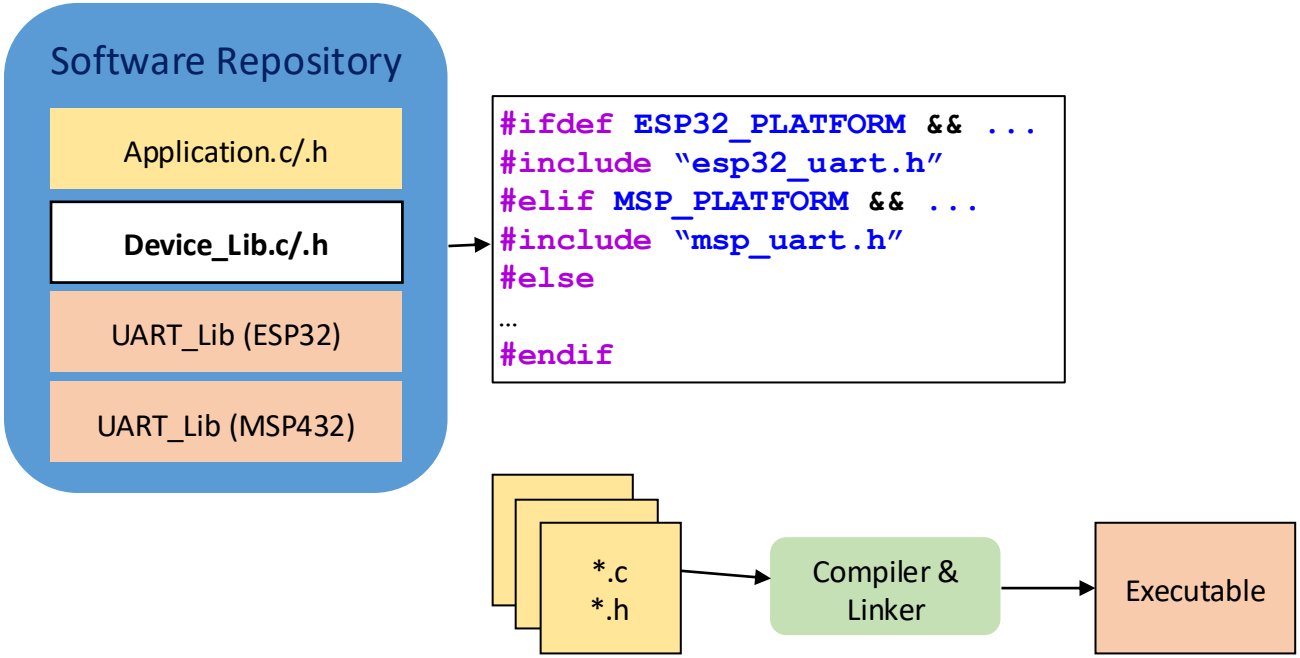
- Condition provided at compile time to dictate **WHAT** should be compiled
  - Useful for debugging and portable code
  - “Turn Off” Large amounts of code

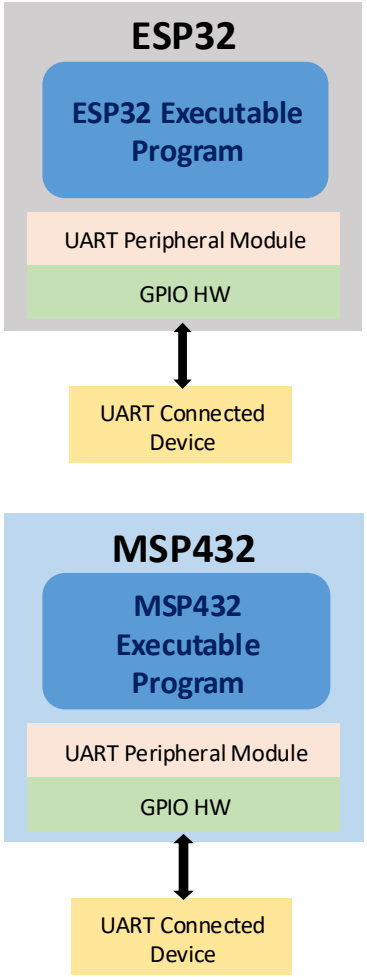
```
#define COMPILE_CODE
…
#ifdef COMPILE_CODE
    // Code will be compiled
#endif
```

```
#define ESP32_PLATFORM

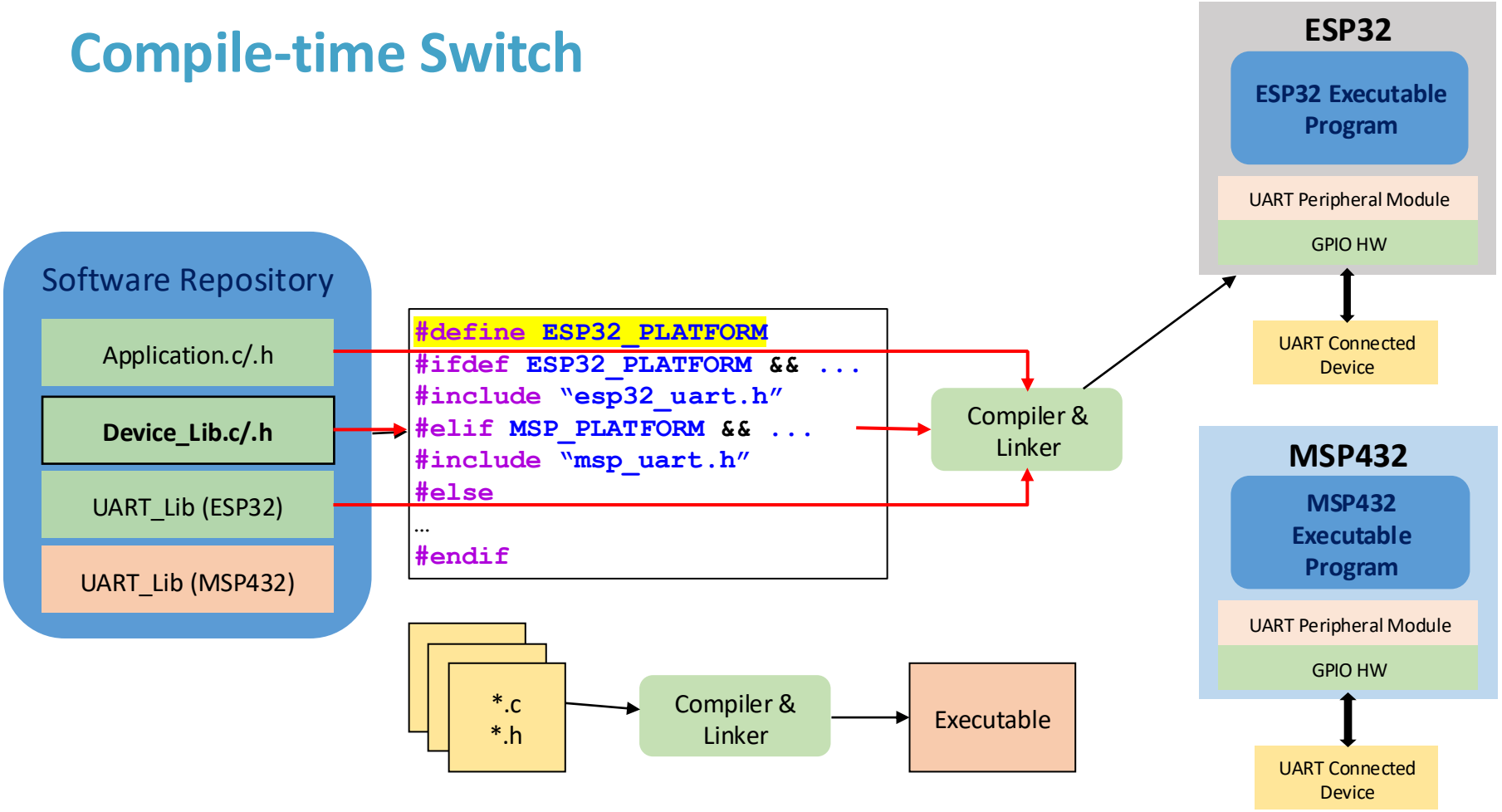
int main(void){
...
#ifdef ( ESP32_PLATFORM ) && ( !MSP_PlATFROM )
 esp32_initialize();
#elif ( MSP_PlATFROM ) && ( !ESP32_PLATFORM )
 msp_initialize();
#else
 #error “Please specify one platform target”
#endif
...
}
```

Compile-time Switch

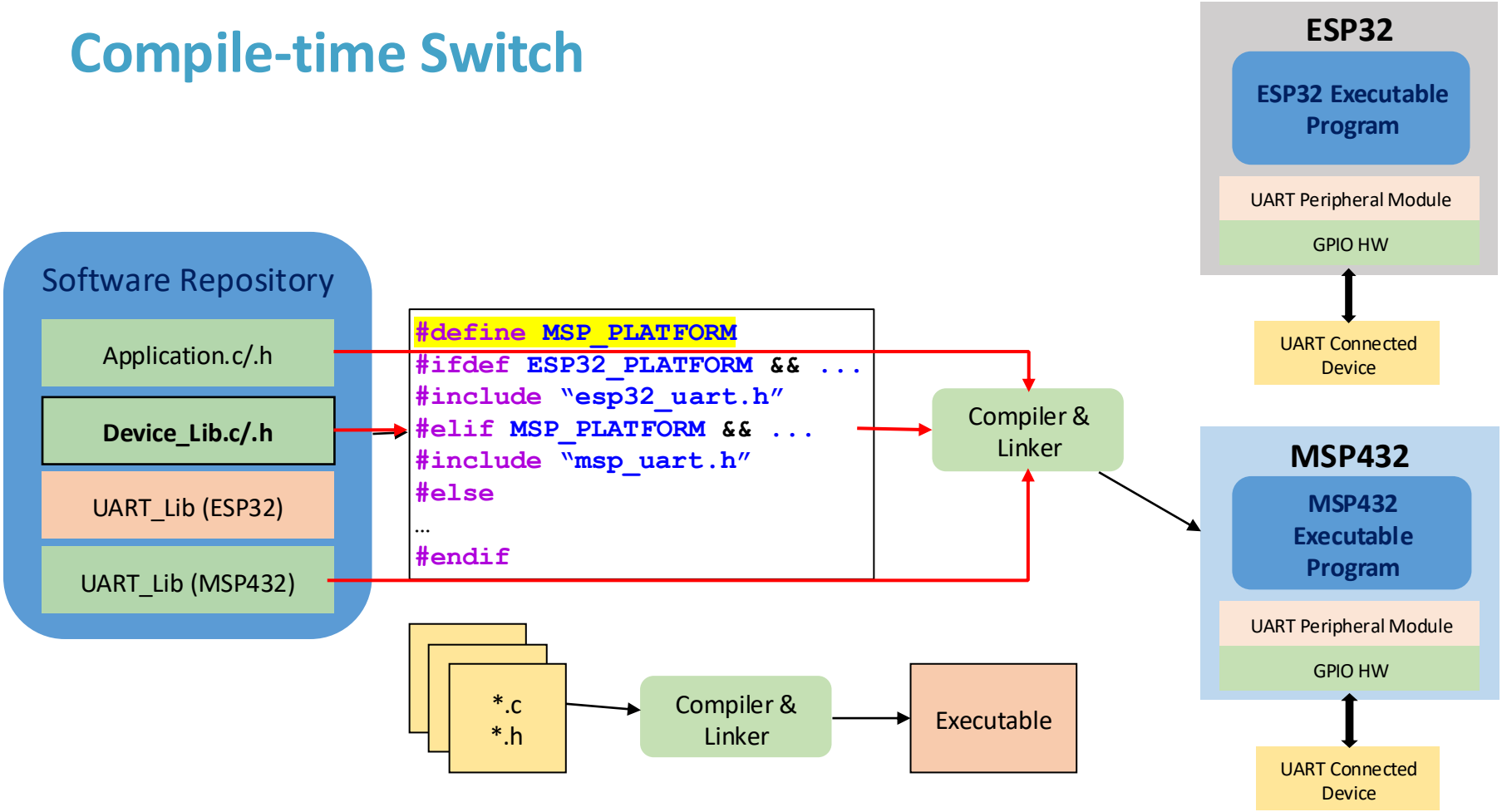




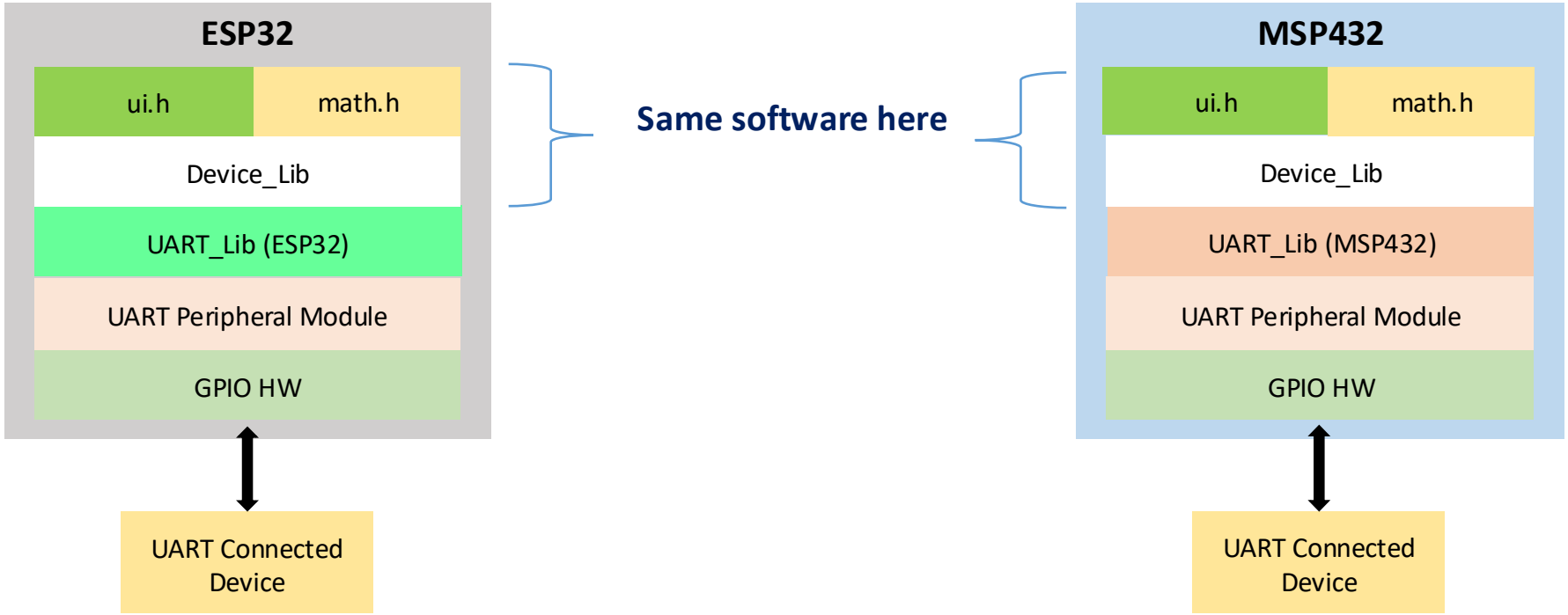
Compile-time Switch



Compile-time Switch

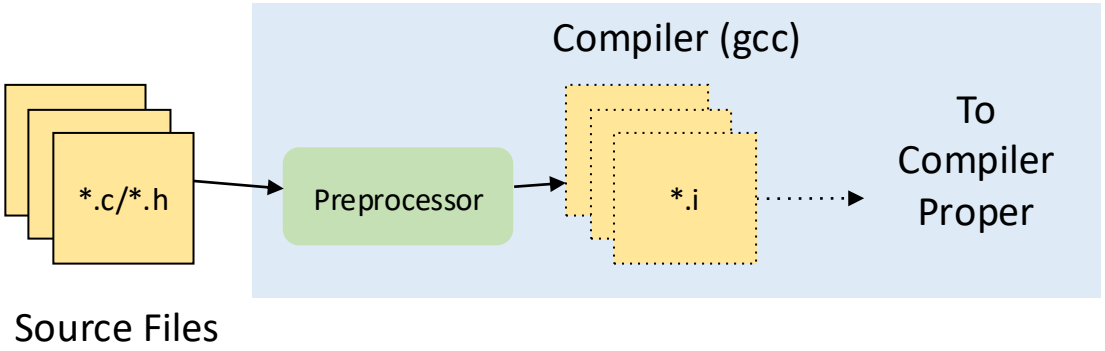


Compile-time Switch - Code Reuse



Compile-time Switch

- Also define platform while compiling



Add extra option to gcc command to define Macro

`-D<MACRO-NAME>`

```
$ gcc -DMSP_PLATFORM -o main.out main.c
```

Pragmas

- Gives a specific instruction to the compiler
  - Controls compilation from software instead of command line
- **Compiler specific**
  - Unrecognized pragmas will be ignored

Pragmas

- Adds options to compiler for specific function

```
#pragma GCC push_options
```

- Causes an error during compilation if code uses these functions

```
#pragma GCC poison printf sprint fprintf
```

- Compile a function with a specific architecture

```
#pragma GCC target (“arch=armv6”)  -or- (“cpu=cortex-m0plus”)
```

Software Modules

- Software organization that each module has encapsulated certain functionality within a library

my_file.c

```
#include "my_file.h"
char arr[LENGTH];

void clear(char * ptr, int size){
int i;
 for(i = 0, i < size, i++){
   ptr[i] = 0;
 }
}
```

my_file.h

```
#define LENGTH (10)
void clear(char * ptr, int size);
```

Preprocessed



my_file.i

```
void clear(char * ptr, int size);

char arr[10];

void clear(char * ptr, int size){
 int i;
 for(i = 0, i < size, i++){
   ptr[i] = 0;
 }
}
```

Software Modules

- Implementation files (*.c):
  - Contains the function definitions or the actual implementation details
- Header files (*.h):
  - Contain the function declarations, macros, & derived data type definitions (structs, enums)

memory.c

```c
#include "memory.h"

char memzero(char * src, int length){
 int i;
 for(i = 0; i < length; i++){
   *src++ = 0;
 }
}
```

memory.h

```c
#ifndef __MEMORY_H__
#define __MEMORY_H__

char memzero(char * src, int length);

#endif /* __MEMORY_H__ */
```

Software Modules

- Top of Header file contains a #ifndef statement
  - Protects against redundant includes and **repeated** declarations

memory.h

```
#ifndef __MEMORY_H__
#define __MEMORY_H__

char memzero(char * src, int length);

#endif /* __MEMORY_H__ */
```

memory.c

```
#include "memory.h"
#include "memory.h"

int main(){
 char arr[10];
 memzero(arr, 10);
 return 0;
}
```

Software Modules

- Header files are the **interface**
  - Anything you want to give access to, put in header file
  - Make Informative function comments in header File
    - Function Description
    - Inputs: type and description
    - Return: type and description

```
#ifndef __MEMORY_H__
#define __MEMORY_H__
/*************************************
***
*  memzero() - Takes a pointer to a
*       location in memory and sets
*       the contents to zero for
*       a length bytes.
*  char * src: Pointer starting byte
*  int length: Number of bytes to zero
*  char (return): Success or Failure
of
*                  operation
*************************************
*/
char memzero(char * src, int length);

#endif /* __MEMORY_H__ */
```

Including Precompiled Libraries

- Questions you Should Ask:
  - If library is precompiled:
    - Is it compiled for my architecture?
    - Was this designed to be optimized for my architecture?
  - If you have full library source code:
    - What software features does this use?
    - What other code does this include?

```
#include <stdlib.h>
#include <math.h>
#include <stdio.h>
#include "thirdparty.h"

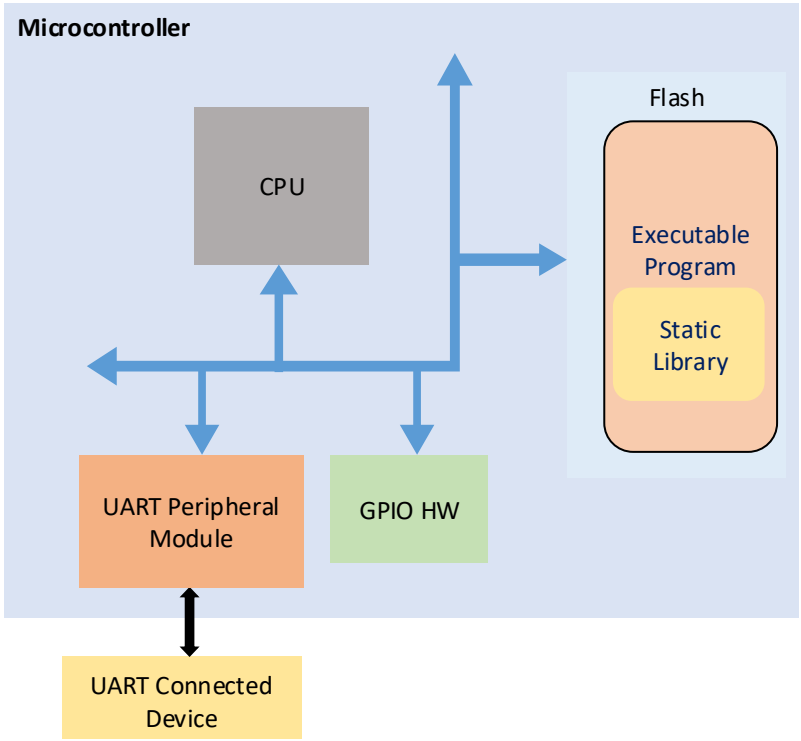
int main(){
  /* Some Code here */

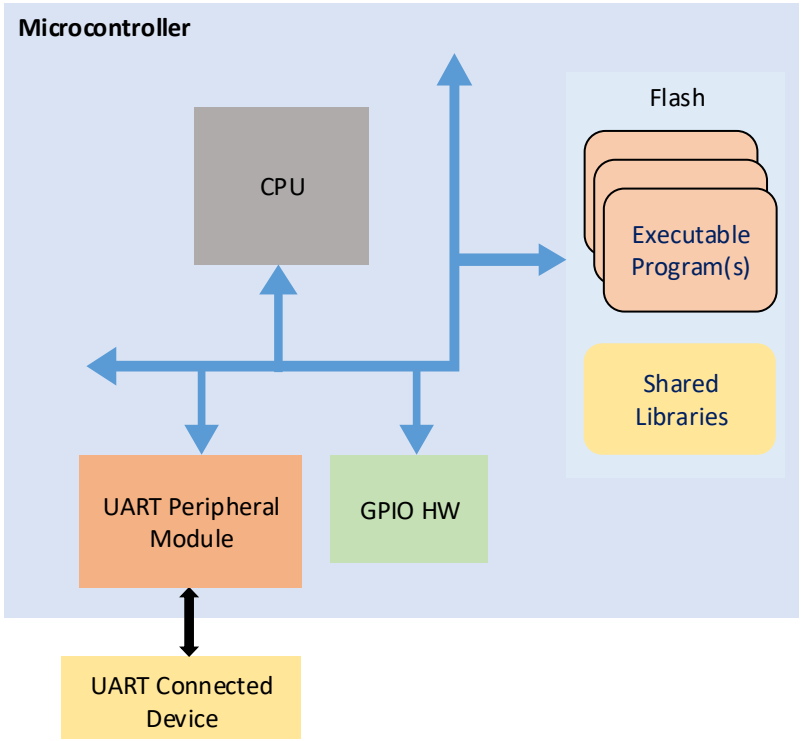
 return 0;
}
```

Compiled Libraries

- **Static Libraries:** Directly linked into your output executable
  - Deployed with the program image as part of the executable
- **Shared libraries:** Linked dynamically at runtime with your executable
  - Pre-installed onto target
  - Used for applications with an operating systems
  - Create with shared flag
  - In general, we do not go for this in embedded systems.

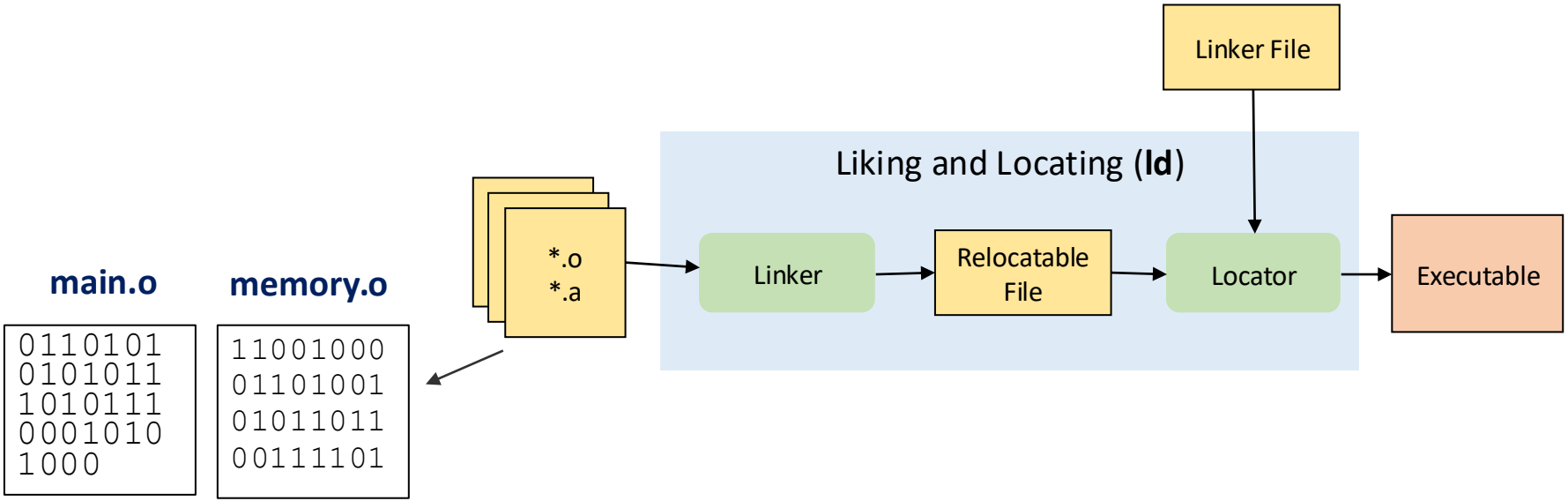
Compiled Libraries





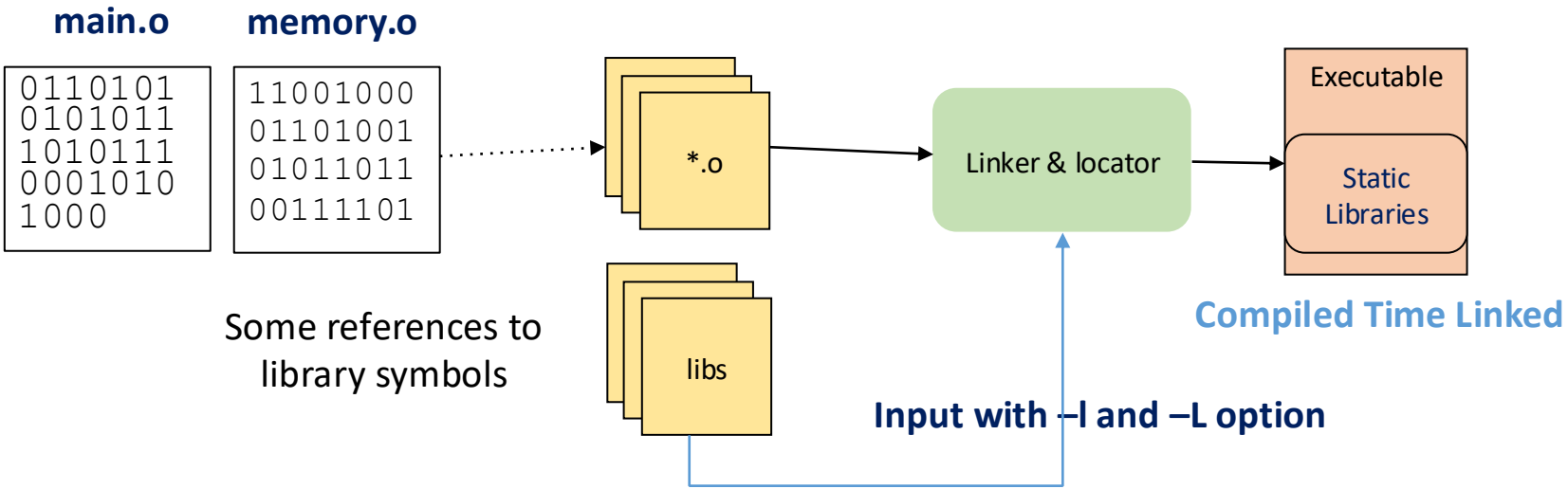
Linkers

- Combines all of objects files into a single executable
  - Object code uses symbols to reference other functions/variables



Linkers and Libraries

- Linker must know name and path to library to link with it
  - Static and Dynamic libraries get linked at different points



Linking Object Files

- Entry and exit points from main are included in a standard library that is automatically included by the linker.
  - Can stop auto link of standard libs with –**nostdlib** flag

How do we enter main?

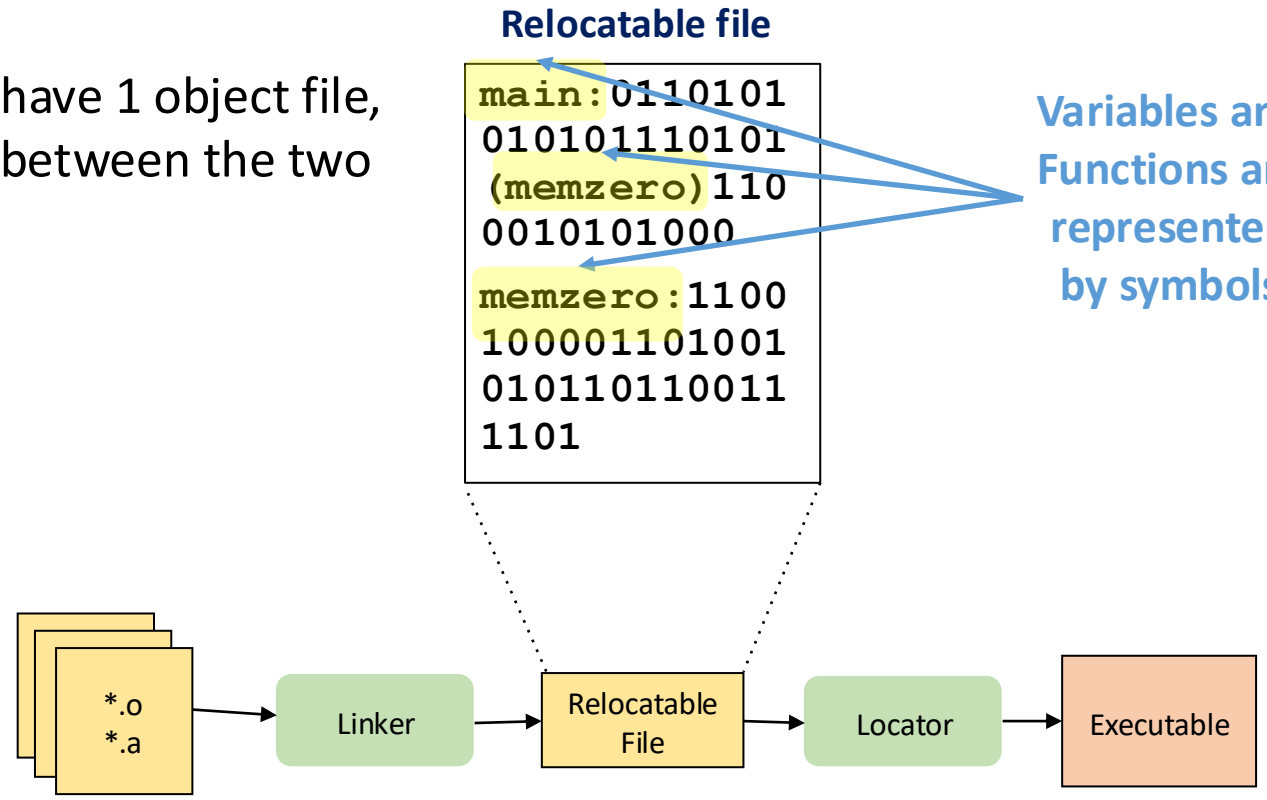
How do we exit or return from main?

```
#include "memory.h"

int main(){
 char arr[10];
 memzero(arr, 10);
 return 0;
}
```

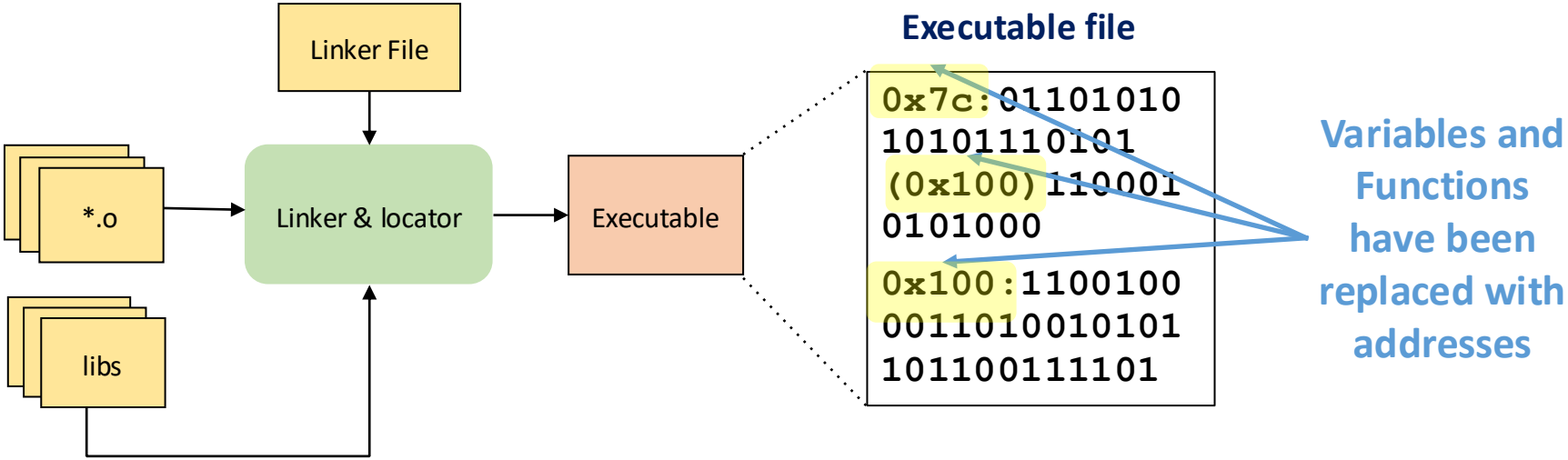
Linking Object Files

- After linking, we have 1 object file, and the symbols between the two are resolved.



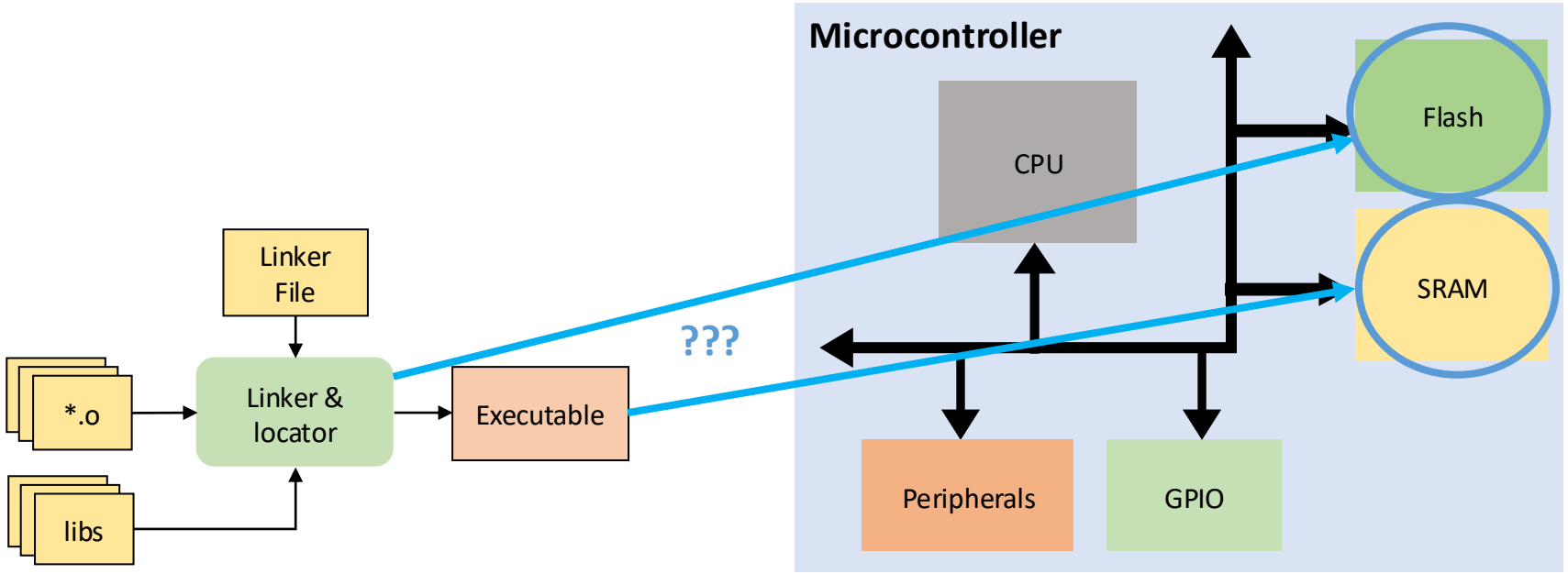
Linking Object Files

- After locating, symbols are removed and direct addresses get assigned into the object code



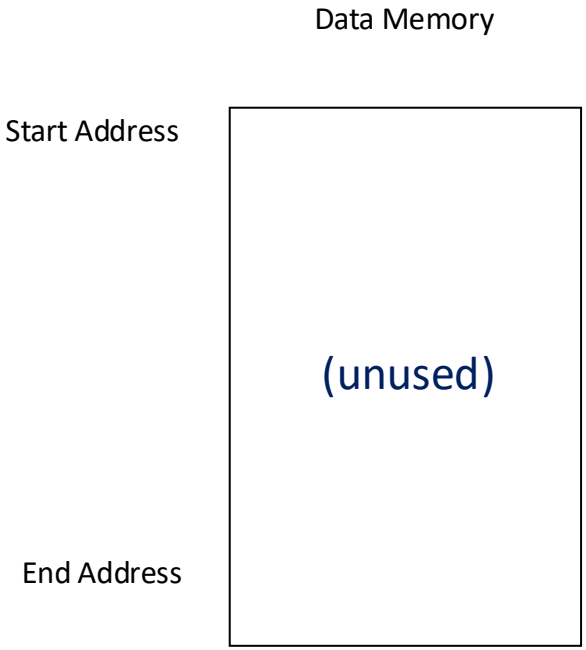
Linker Files

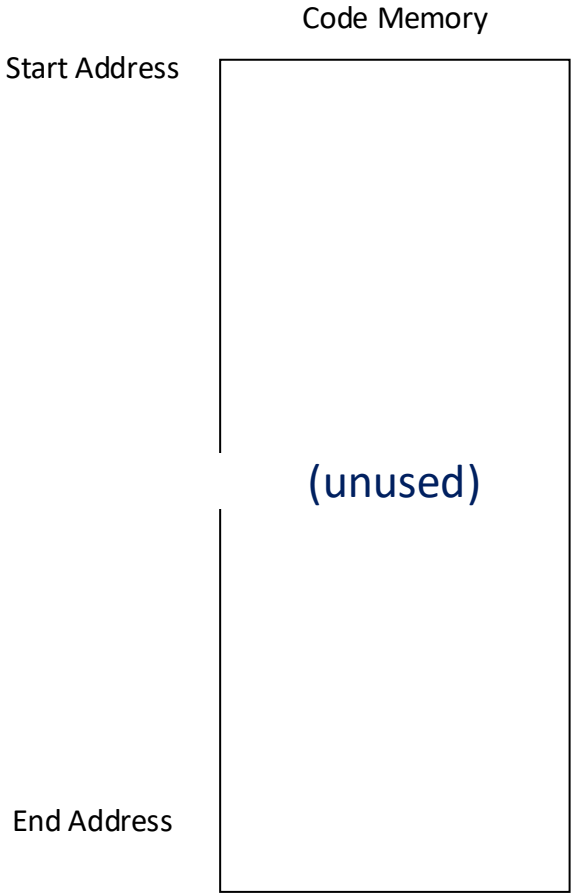
- Details on how to map compiled data into physical memory regions



Linker Files

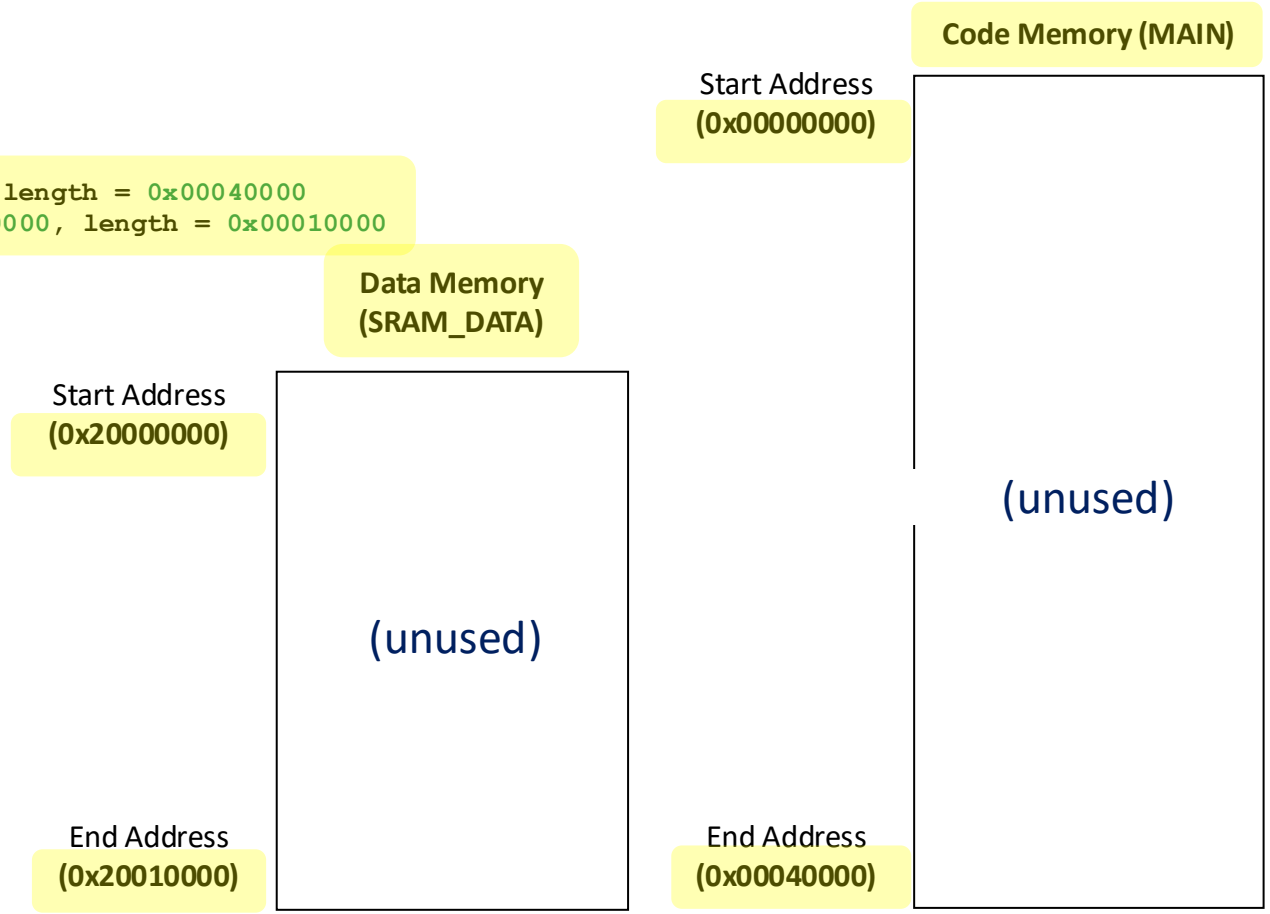
```
MEMORY
{
 MAIN (RX) : origin = 0x00000000, length = 0x00040000
 SRAM_DATA (RW) : origin = 0x20000000, length = 0x00010000
}
SECTIONS
{
 .intvecs:   > 0x00000000
 .text    :   > MAIN
 .const   :   > MAIN
 .cinit   :   > MAIN
 .pinit   :   > MAIN
 .data    :   > SRAM_DATA
 .bss     :   > SRAM_DATA
 .heap    :   > SRAM_DATA
 .stack   :   > SRAM_DATA (HIGH)
}
```





Linker Files

```
MEMORY
{
 MAIN (RX) : origin = 0x00000000, length = 0x00040000
 SRAM_DATA (RW) : origin = 0x20000000, length = 0x00010000
}
SECTIONS
{
 .intvecs:   > 0x00000000
 .text    :   > MAIN
 .const   :   > MAIN
 .cinit   :   > MAIN
 .pinit   :   > MAIN
 .data    :   > SRAM_DATA
 .bss     :   > SRAM_DATA
 .heap    :   > SRAM_DATA
 .stack   :   > SRAM_DATA (HIGH)
}
```



Linker Files

```
MEMORY
{
 MAIN (RX) : origin = 0x00000000, length = 0x00040000
 SRAM_DATA (RW) : origin = 0x20000000, length = 0x00010000
}
SECTIONS
{
 .intvecs:   > 0x00000000
 .text    :   > MAIN
 .const   :   > MAIN
 .cinit   :   > MAIN
 .pinit   :   > MAIN
 .data    :   > SRAM_DATA
 .bss     :   > SRAM_DATA
 .heap    :   > SRAM_DATA
 .stack   :   > SRAM_DATA (HIGH)
}
```

